PHISHING URL DETECTOR

ABSTRACT

Phishing attacks are one of the most common cyber threats used by attackers to steal sensitive information such as usernames, passwords, banking details, and personal data. Many phishing websites imitate legitimate websites to deceive users. This project presents a Rule-Based Phishing URL Detector developed using Python. The system analyzes a given URL and identifies suspicious characteristics such as phishing keywords, excessive subdomains, IP-based URLs, lack of HTTPS, and typosquatting attempts. Based on the detected indicators, the URL is classified as Safe, Suspicious, or Phishing. The project demonstrates how simple cybersecurity techniques can help users identify potentially dangerous websites before interacting with them.

INTRODUCTION

The rapid growth of internet usage has increased the number of cyberattacks targeting users worldwide. Phishing is a cybercrime technique where attackers create fake websites that resemble legitimate websites in order to steal sensitive information.

Many users fail to recognize phishing websites due to their realistic appearance. Therefore, an automated detection mechanism is necessary to identify suspicious URLs before users access them.

This project focuses on developing a Python-based phishing detection tool that evaluates URLs using predefined rules and generates a risk score to determine whether a URL is safe or potentially malicious.

PROBLEM STATEMENT

Users often encounter malicious URLs through emails, messages, advertisements, and social media platforms. Identifying phishing websites manually is difficult for non-technical users.

The objective of this project is to develop a system capable of detecting phishing URLs using rule-based analysis techniques and alerting users about potential threats.

OBJECTIVES

- To analyze URLs for phishing indicators.
- To detect suspicious keywords commonly used in phishing attacks.
- To verify whether a URL uses HTTPS.
- To identify IP-based URLs.
- To detect typosquatting and misspellings.
- To classify URLs as Safe, Suspicious, or Phishing.

TECHNOLOGIES USED

1. **Programming Language**

- Python

2. **Libraries Used**

- re (Regular Expressions)

3. **Development Environment**

- Visual Studio Code
- Command Prompt

SYSTEM REQUIREMENTS

1.Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB minimum
- Storage: 100 MB free space

2.Software Requirements

- Windows 10/11
- Python 3.x
- VS Code

METHODOLOGY

The phishing detection process follows the steps below:

1. User enters a URL.
2. URL is converted to lowercase.
3. System checks for suspicious keywords.
4. System verifies URL length.
5. System detects '@' symbol usage.
6. System checks for IP addresses.
7. System verifies HTTPS usage.
8. System identifies misspellings and typosquatting.
9. System analyzes subdomain count.

10. Risk score is calculated.
11. URL is classified as Safe, Suspicious, or Phishing.
12. Reasons for classification are displayed.

ALGORITHM

Step 1: Accept URL from user.

Step 2: Initialize phishing score as zero.

Step 3: Check for suspicious keywords.

Step 4: Check URL length.

Step 5: Check for '@' symbol.

Step 6: Detect IP address usage.

Step 7: Verify HTTPS protocol.

Step 8: Detect possible misspellings.

Step 9: Count subdomains.

Step 10: Update risk score accordingly.

Step 11: Classify URL based on score.

Step 12: Display result and reasons.

IMPLEMENTATION

The system uses rule-based detection techniques.

Detection Rules

1. Suspicious Keywords
    - login
    - verify
    - secure
    - account
    - update
    - bank
    - free
    - password

- confirm

2. Long URL Detection
    - URLs longer than 75 characters are flagged.
3. '@' Symbol Detection
    - Attackers often use '@' to disguise destinations.
4. IP Address Detection
    - URLs containing direct IP addresses are considered suspicious.
5. HTTPS Verification
    - URLs not using HTTPS are penalized.
6. Typosquatting Detection
    - Detects characters such as 0, 1, and @.
7. Subdomain Analysis
    - Excessive subdomains indicate suspicious behavior.

SAMPLE OUTPUTS

Example 1

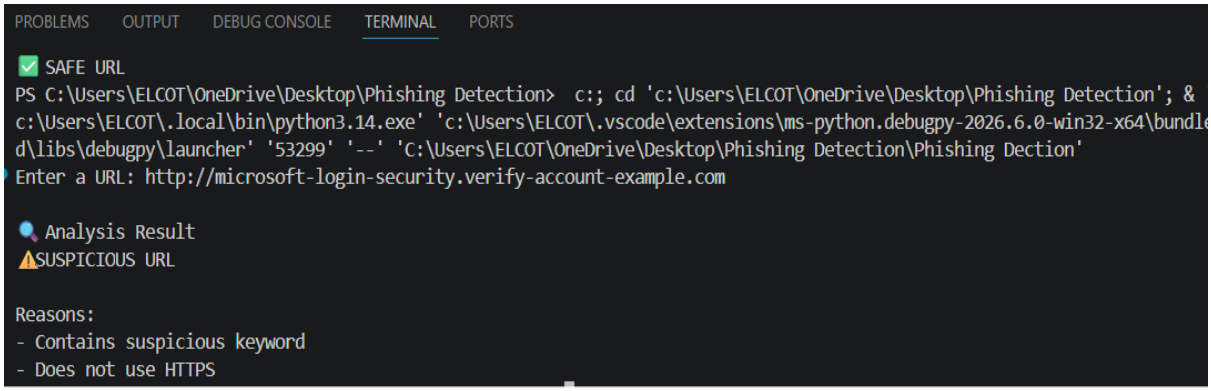
Input:
https://www.microsoft.com/

Output:
SAFE URL

Example 2

Input:
http://microsoft-login-security.verify-account-example.com/

Output:
SUSPICIOUS URL



ADVANTAGES

- Easy to implement.
- Fast URL analysis.
- Lightweight application.
- No internet connection required.
- User-friendly operation.
- Helps increase cybersecurity awareness.

LIMITATIONS

- Rule-based detection may generate false positives.
- Cannot detect all advanced phishing attacks.
- Does not verify domain reputation.
- Does not use machine learning techniques.

FUTURE ENHANCEMENTS

- Machine Learning based phishing detection.
- Real-time URL scanning.
- Domain reputation checking.
- WHOIS lookup integration.

- Browser extension implementation.
- Threat intelligence integration.
- Graphical User Interface (GUI).

APPLICATIONS

- Cybersecurity awareness training.
- Educational projects.
- Security research.
- URL verification systems.
- Beginner cybersecurity tools.

RESULTS

The developed phishing URL detector successfully identifies suspicious URLs based on predefined phishing indicators. The system effectively classifies URLs into Safe, Suspicious, and Phishing categories while providing clear explanations for the detected threats.

CONCLUSION

Phishing attacks continue to be a major cybersecurity challenge. The developed Rule-Based Phishing URL Detector provides a simple and effective solution for identifying potentially malicious URLs. By analyzing multiple phishing indicators and calculating a risk score, the system helps users recognize suspicious websites and improve their online safety. The project demonstrates the practical application of Python programming and cybersecurity concepts in detecting phishing threats.

REFERENCES

1. Python Official Documentation
2. OWASP Phishing Prevention Guidelines
3. Cybersecurity and Infrastructure Security Agency (CISA)
4. Regular Expressions Documentation
5. Cybersecurity Research Articles on Phishing Detection